

Списки

В предыдущих уроках мы работали с последовательностями чисел, символов, строк, но не сохраняли всю последовательность в памяти компьютера, а обрабатывали ее поэлементно, считывая раз за разом новый элемент. Однако во многих задачах требуется **сохранять всю последовательность**. Например, классическая задача [сортировки](#) (упорядочения) некоторой последовательности требует сохранения всех данных в памяти компьютера. Увы, не сохранив, их невозможно отсортировать. И тут на помощь приходит структура данных, которая в большинстве языков программирования называется [массивом](#). В Python она называется **списком**.

Структура данных (data structure) — программная единица, позволяющая **хранить и обрабатывать** множество однотипных и/или логически связанных данных.

Список представляет собой последовательность элементов, пронумерованных от 0, как символы в строке.

Создание списка

Чтобы создать список, нужно перечислить его элементы через запятую в квадратных скобках:

```
numbers = [2, 4, 6, 8, 10]
```

```
languages = ['Python', 'C#', 'C++', 'Java']
```

Список `numbers` состоит из 5 элементов, и каждый из них — целое число.

- `numbers[0] == 2;`
- `numbers[1] == 4;`
- `numbers[2] == 6;`
- `numbers[3] == 8;`
- `numbers[4] == 10.`

Список `languages` состоит из 4 элементов, каждый из которых — **строка**.

- `languages[0] == 'Python';`
- `languages[1] == 'C#';`
- `languages[2] == 'C++';`
- `languages[3] == 'Java'.`

Значения, заключенные в квадратные скобки и отделенные запятыми, называются **элементами списка**.

Список может содержать значения **разных типов данных**:

```
info = ['Timur', 1992, 61.5]
```

Список `info` содержит строковое значение, целое число и число с плавающей точкой.

- `info[0] == 'Timur';`
- `info[1] == 1992;`
- `info[2] == 61.5.`

Обычно элементы списка содержат данные одного типа и на практике редко приходится создавать списки содержащие элементы разных типов данных.

Пустой список

Создать пустой список можно двумя способами:

1. Использовать пустые квадратные скобки `[]`;
2. Использовать встроенную функцию, которая называется `list`.

Следующие две строки кода создают пустой список:

```
mylist = [] # пустой список  
mylist = list() # пустой список
```

Вывод списка

Для вывода всего списка можно применить функцию `print()`:

```
numbers = [2, 4, 6, 8, 10]  
  
languages = ['Python', 'C#', 'C++', 'Java']  
  
print(numbers)  
  
print(languages)
```

Функция `print()` выводит на экран элементы списка, в квадратных скобках, разделенные запятыми:

```
[2, 4, 6, 8, 10]  
  
['Python', 'C#', 'C++', 'Java']
```

Обратите внимание, что вывод списка содержит квадратные скобки. Позже мы научимся выводить элементы списка в более удобном виде с помощью циклов.

Встроенная функция list

Python имеет встроенную функцию `list()`, которая помимо создания пустого списка может преобразовывать некоторые типы объектов в списки.

Например, мы знаем, что функция `range()` создает последовательность целых чисел в заданном диапазоне. Для преобразования этой последовательности в список, мы пишем следующий код:

```
numbers = list(range(5))
```

Во время исполнения этого кода происходит следующее:

1. Вызывается функция `range()`, в которую в качестве аргумента передается число 5;
2. Эта функция возвращает последовательность чисел `0, 1, 2, 3, 4`;
3. Последовательность чисел `0, 1, 2, 3, 4` передается в качестве аргумента в функцию `list()`;
4. Функция `list()` возвращает список `[0, 1, 2, 3, 4]`;
5. Список `[0, 1, 2, 3, 4]` присваивается переменной `numbers`.

Вот еще один пример:

```
even_numbers = list(range(0, 10, 2)) # список содержит четные числа  
0, 2, 4, 6, 8
```

```
odd_numbers = list(range(1, 10, 2)) # список содержит нечетные числа  
1, 3, 5, 7, 9
```

Точно также с помощью функции `list()` мы можем создать список из символов строки. Для преобразования строки в список мы пишем следующий код:

```
s = 'abcde'
```

```
chars = list(s) # список содержит символы 'a', 'b', 'c', 'd', 'e'
```

Во время исполнения этого кода происходит следующее:

1. Вызывается функция `list()`, в которую в качестве аргумента передается строка `'abcde'`;
2. Функция `list()` возвращает список `['a', 'b', 'c', 'd', 'e']`;
3. Список `['a', 'b', 'c', 'd', 'e']` присваивается переменной `chars`.

Примечания

Примечание 1. Как уже было сказано, списки в Python аналогичны массивам в других языках программирования. Однако разница между списками и массивами все же существует. Элементы массива всегда имеют одинаковый тип данных и располагаются в памяти компьютера непрерывным блоком, а элементы списка могут быть разбросаны по памяти как угодно и могут иметь разный тип данных.

Примечание 2. Обратите внимание, при выводе содержимого списка с помощью функции `print()`, все строковые элементы списка обрамляются **одинарными кавычками**. Если требуется осуществить вывод в двойных кавычках, нужно самостоятельно писать код вывода.

Основы работы со списками

Работа со списками очень сильно напоминает работу со строками, поскольку и списки, и строки содержат отдельные элементы: элементы списка могут иметь произвольный тип, а элементами строк всегда являются символы. Многие из того, что мы делали со строками, доступно и при работе со списками.

Функция len()

Длиной списка называется количество его элементов. Для того, чтобы посчитать длину списка мы используем встроенную функцию `len()` (от слова length – длина).

Следующий программный код:

```
numbers = [2, 4, 6, 8, 10]
```

```
languages = ['Python', 'C#', 'C++', 'Java']
```

```
print(len(numbers))      # выводим длину списка numbers
```

```
print(len(languages))    # выводим длину списка languages
```

```
print(len(['apple', 'banana', 'cherry'])) # выводим длину списка,  
состоящего из 3 элементов
```

выведет:

```
5
```

```
4
```

```
3
```

Оператор принадлежности in

Оператор `in` позволяет проверить, содержит ли список некоторый элемент.

Рассмотрим следующий код:

```
numbers = [2, 4, 6, 8, 10]
```

```
if 2 in numbers:
```

```
    print('Список numbers содержит число 2')
```

```
else:  
    print('Список numbers не содержит число 2')
```

Такой код проверяет, содержит ли список `numbers` число 2 и выводит соответствующий текст:

Список `numbers` содержит число 2

Мы можем использовать оператор `in` вместе с логическим оператором `not`. Например

```
numbers = [2, 4, 6, 8, 10]
```

```
if 0 not in numbers:  
    print('Список numbers не содержит нулей')
```

Индексация

При работе со строками мы использовали **индексацию**, то есть обращение к конкретному символу строки по его индексу. Аналогично, можно индексировать и списки.

Для индексации списков в Python используются квадратные скобки `[]`, в которых указывается индекс (номер) нужного элемента в списке:

Пусть `numbers = [2, 4, 6, 8, 10]`.

Таблица ниже, показывает как работает индексация:

Выражение	Результат	Пояснение
<code>numbers[0]</code>	2	первый элемент списка
<code>numbers[1]</code>	4	второй элемент списка
<code>numbers[2]</code>	6	третий элемент списка
<code>numbers[3]</code>	8	четвертый элемент списка
<code>numbers[4]</code>	10	пятый элемент списка

Обратите внимание первый элемент списка `numbers[0]`, а не `numbers[1]`.

Так же, как и в строках, для нумерации с конца разрешены отрицательные индексы.

Выражение	Результат	Пояснение
<code>numbers[-1]</code>	10	пятый элемент списка
<code>numbers[-2]</code>	8	четвертый элемент списка
<code>numbers[-3]</code>	6	третий элемент списка
<code>numbers[-4]</code>	4	второй элемент списка
<code>numbers[-5]</code>	2	первый элемент списка

Как и в строках, попытка обратиться к элементу списка по несуществующему индексу:

```
print(numbers[17])
```

вызовет ошибку:

```
IndexError: index out of range
```

Срезы

Рассмотрим список `numbers = [2, 4, 6, 8, 10]`.

С помощью среза мы можем получить несколько элементов списка, создав диапазон индексов разделенных двоеточием `numbers[x:y]`.

Следующий программный код:

```
print(numbers[1:3])
```

```
print(numbers[2:5])
```

ВЫВОДИТ:

```
[4, 6]
```

```
[6, 8, 10]
```

При построении среза `numbers[x:y]` первое число – это то место, где начинается срез (**включительно**), а второе – это место, где заканчивается срез (**невключительно**). Разрезая списки, мы создаем новые списки, по сути, подсписки исходного.

При использовании срезов со списками мы также можем опускать второй параметр в срезе `numbers[x:]` (но поставить двоеточие), тогда срез берется до конца списка. Аналогично если опустить первый параметр `numbers[:y]`, то можно взять срез от начала списка.

Срез `numbers[:]` возвращает копию исходного списка.

Как и в строках, мы можем использовать отрицательные индексы в срезах списков.

Использование срезов для изменения элементов в заданном диапазоне

Для изменения целого диапазона элементов списка можно использовать срезы. Например, если мы хотим перевести на русский язык названия фруктов `'banana', 'cherry', 'kiwi'`, то это можно сделать с помощью среза.

Следующий программный код:

```
fruits = ['apple', 'apricot', 'banana', 'cherry', 'kiwi', 'lemon', 'mango']
```

```
fruits[2:5] = ['банан', 'вишня', 'киви']
```

```
print(fruits)
```

ВЫВОДИТ:

```
['apple', 'apricot', 'банан', 'вишня', 'киви', 'lemon', 'mango']
```

Операция конкатенации + и умножения на число *

Мы можем применять операторы `+` и `*` для списков подобно тому как мы это делали со строками.

Следующий программный код:

```
print([1, 2, 3, 4] + [5, 6, 7, 8])
```

```
print([7, 8] * 3)
```

```
print([0] * 10)
```

ВЫВОДИТ:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

```
[7, 8, 7, 8, 7, 8]
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Для генерации списков, состоящих строго из повторяющихся элементов, умножение на число — самый короткий и правильный метод.

Мы также можем использовать расширенные операторы `+=` и `*=` при работе со списками.

Следующий программный код:

```
a = [1, 2, 3, 4]
b = [7, 8]

a += b    # добавляем к списку a список b
b *= 5    # повторяем список b 5 раз

print(a)
print(b)
```

ВЫВОДИТ:

```
[1, 2, 3, 4, 7, 8]
[7, 8, 7, 8, 7, 8, 7, 8, 7, 8]
```

Отличие списков от строк

Несмотря на всю схожесть списков и строк, есть одно очень важное отличие: строки — **неизменяемые** объекты, а списки — **изменяемые**.

Следующий программный код:

```
s = 'abcdefg'
s[1] = 'x'    # пытаемся изменить 2 символ (по индексу 1) строки
```

приводит к ошибке:

```
object does not support item assignment
```

Следующий программный код:

```
numbers = [1, 2, 3, 4, 5, 6, 7]
numbers[1] = 101    # изменяем 2 элемент (по индексу 1) списка
print(numbers)
```

ВЫВОДИТ:

```
[1, 101, 3, 4, 5, 6, 7]
```

Запомни: изменять отдельные символы строк нельзя, однако можно изменять отдельные элементы списков. Для этого используем индексатор и оператор присваивания.

